

SKILLWEEK11

TASK1: To connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

```
sanjay_dundra@Sanjay: ~/ × + ▼
GNU nano 7.2 task1.c *
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int subtract(int a, int b);

#endif
```

```
sanjay_dundra@Sanjay: ~/ × + ▼
GNU nano 7.2 task1.c *
#include "calculator.h"

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}
```

```
sanjay_dundra@Sanjay: ~/ × + ▼
GNU nano 7.2 main.c *
#include <stdio.h>
#include "calculator.h"

int main()
{
    int a = 10;
    int b = 5;

    printf("Addition = %d\n", add(a, b));
    printf("Subtraction = %d\n", subtract(a, b));

    return 0;
}
```

```
sanjay_dundra@Sanjay: ~ × + ▼ - □ ×
sanjay_dundra@Sanjay:~$ mkdir Skill11
sanjay_dundra@Sanjay:~$ cd Skill11
sanjay_dundra@Sanjay:~/Skill11$ nano calculator.h
sanjay_dundra@Sanjay:~/Skill11$ nano calculator.c
sanjay_dundra@Sanjay:~/Skill11$ nano main.c
sanjay_dundra@Sanjay:~/Skill11$ gcc main.c calculator.c -o program
sanjay_dundra@Sanjay:~/Skill11$ ./program
Addition = 15
Subtraction = 5
sanjay_dundra@Sanjay:~/Skill11$ |
```

TASK2: To detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

```
sanjay_dundra@Sanjay: ~/ × + ∨  
GNU nano 7.2 error_handling.c *  
#include <stdio.h>  
  
int main()  
{  
    int a, b;  
  
    printf("Enter two integers: ");  
  
    if (scanf("%d %d", &a, &b) != 2)  
    {  
        printf("ERROR: Invalid input. Please enter integers.\n");  
        return 1;  
    }  
  
    if (b == 0)  
    {  
        printf("ERROR: Division by zero is not allowed.\n");  
        return 1;  
    }  
  
    printf("Result = %d\n", a / b);  
  
    return 0;  
}
```

```
sanjay_dundra@Sanjay:~/Skill11$ nano error_handling.c  
sanjay_dundra@Sanjay:~/Skill11$ gcc error_handling.c -o error_handling  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling  
Enter two integers: 10 2  
Result = 5  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling  
Enter two integers: 10 4  
Result = 2  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling  
Enter two integers: 10 0  
ERROR: Division by zero is not allowed.  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling  
Enter two integers: abc xyz  
ERROR: Invalid input. Please enter integers.  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling > output.log 2>&1  
sanjay_dundra@Sanjay:~/Skill11$ cat output.log  
Enter two integers: ERROR: Invalid input. Please enter integers.  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling > output.log 2>&1  
sanjay_dundra@Sanjay:~/Skill11$ cat output.log  
Enter two integers: 10 2  
Result = 5  
sanjay_dundra@Sanjay:~/Skill11$ ./error_handling 2> error.log  
og  
Enter two integers: 8 4  
Result = 2  
sanjay_dundra@Sanjay:~/Skill11$ cat error.log
```

TASK3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

```
sanjay_dundra@Sanjay: ~/ × + ▼
GNU nano 7.2 memory.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr;
    arr = malloc(5 * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
    {
        arr[i] = i + 1;
    }

    printf("Values:\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");
    return 0;
}
```

```
sanjay_dundra@Sanjay: ~/ × + ▼
sanjay_dundra@Sanjay: ~/Skill111$ nano memory.c
sanjay_dundra@Sanjay: ~/Skill111$ gcc -g memory.c -o memory
sanjay_dundra@Sanjay: ~/Skill111$ ./memory
Values:
1 2 3 4 5
sanjay_dundra@Sanjay: ~/Skill111$ valgrind --leak-check=full ./memory
==662== Memcheck, a memory error detector
==662== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==662== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==662== Command: ./memory
==662==
Values:
1 2 3 4 5
==662==
==662== HEAP SUMMARY:
==662==   in use at exit: 20 bytes in 1 blocks
==662==   total heap usage: 2 allocs, 1 frees, 1,044 bytes allocated
==662==
==662== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==662==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==662==    by 0x1091BE: main (memory.c:8)
==662==
==662== LEAK SUMMARY:
==662==   definitely lost: 20 bytes in 1 blocks
==662==   indirectly lost: 0 bytes in 0 blocks
==662==   possibly lost: 0 bytes in 0 blocks
==662==   still reachable: 0 bytes in 0 blocks
==662==   suppressed: 0 bytes in 0 blocks
```

```

    {
        arr[i] = i + 1;
    }

    printf("Values:\n");
    for (int i = 0; i < 5; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");
    free(arr);
    return 0;
}
```

```

sanjay_dundra@Sanjay:~/Skill111$ nano memory.c
sanjay_dundra@Sanjay:~/Skill111$ gcc -g memory.c -o memory
sanjay_dundra@Sanjay:~/Skill111$ valgrind --leak-check=full ./memory
==673== Memcheck, a memory error detector
==673== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==673== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright
==673== Command: ./memory
Values:
1 2 3 4 5
==673== HEAP SUMMARY:
==673==   in use at exit: 0 bytes in 0 blocks
==673==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==673== All heap blocks were freed -- no leaks are possible
==673== For lists of detected and suppressed errors, rerun with: -s
==673== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
sanjay_dundra@Sanjay:~/Skill111$ valgrind --leak-check=full ./memory
ory > valgrind_report.txt 2>&1
sanjay_dundra@Sanjay:~/Skill111$ cat valgrind_report.txt
==674== Memcheck, a memory error detector
==674== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==674== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright
==674== Command: ./memory

```

TASK4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

```

GNU nano 7.2 test.sh *
#!/bin/bash

echo "==== TEST 1 ====="
echo "10 2" | ./error_handling

echo ""
echo "==== TEST 2 ====="
echo "10 0" | ./error_handling

echo ""
echo "==== TEST 3 ====="
echo "abc xyz" | ./error_handling

echo ""
echo "==== TESTING COMPLETED ====="

```

```

sanjay_dundra@Sanjay:~/Skill111$ nano test.sh
sanjay_dundra@Saanjay:~/Skill111$ chmod +x test.sh
sanjay_dundra@Sannjay:~/Skill111$ ./test.sh
==== TEST 1 ====
Enter two integers: Result = 5

==== TEST 2 ====
Enter two integers: ERROR: Division by zero is not allowed.

==== TEST 3 ====
Enter two integers: ERROR: Invalid input. Please enter integers.

==== TESTING COMPLETED =====

```


